

4_10xMouse_RA_comparison_geneIntersection

August 31, 2026

```
[1]: library(rtracklayer)
library(ggplot2)
library(Seurat)

getwd()
dir.create("figures_10xMouse_RA")
dir.create("data")
colorDict = c("2CLC"="#88535A",
              "Zscan4+"="#EF8264",
              "Pluri"="#F2CC8F")

colorAge = c("old"="#A58065", #A67C63, #DB9D74
             "young"="#7FCFF2") #049DD9

colorTools = c("SoloTE"="#A4DD9B", #6fc69d",
               "Stellarscope"="#4f5d93",
               "STARsolo"="#f0df93")

dataset_id <- "10xMouse_RA"
```

Loading required package: GenomicRanges

Loading required package: stats4

Loading required package: BiocGenerics

Attaching package: 'BiocGenerics'

The following objects are masked from 'package:stats':

IQR, mad, sd, var, xtabs

The following objects are masked from 'package:base':

anyDuplicated, aperm, append, as.data.frame, basename, cbind,

```
colnames, dirname, do.call, duplicated, eval, evalq, Filter, Find,
get, grep, grepl, intersect, is.unsorted, lapply, Map, mapply,
match, mget, order, paste, pmax, pmax.int, pmin, pmin.int,
Position, rank, rbind, Reduce, rownames, sapply, setdiff, sort,
table, tapply, union, unique, unsplit, which.max, which.min
```

Loading required package: S4Vectors

Attaching package: 'S4Vectors'

The following object is masked from 'package:utils':

```
findMatches
```

The following objects are masked from 'package:base':

```
expand.grid, I, unname
```

Loading required package: IRanges

Loading required package: GenomeInfoDb

Loading required package: SeuratObject

Loading required package: sp

Attaching package: 'sp'

The following object is masked from 'package:IRanges':

```
%over%
```

Attaching package: 'SeuratObject'

The following object is masked from 'package:GenomicRanges':

```
intersect
```

The following object is masked from 'package:GenomeInfoDb':

```
intersect
```

The following object is masked from 'package:IRanges':

```
intersect
```

The following object is masked from 'package:S4Vectors':

```
intersect
```

The following object is masked from 'package:BiocGenerics':

```
intersect
```

The following objects are masked from 'package:base':

```
intersect, t
```

```
~/mnt/TEdata/TEbenchmarking/jupyterNotebooks/realDatasets'
```

```
Warning message in dir.create("figures_10xMouse_RA"):
```

```
"'figures_10xMouse_RA' already exists"
```

```
Warning message in dir.create("data"):
```

```
"'data' already exists"
```

```
[2]: # theme_paper <- function(base_size = 17, base_family = "") {  
#   theme_minimal(base_size = base_size, base_family = base_family) +  
#     theme(  
#       plot.title = element_text(face = "bold", size = base_size + 2, hjust = 0.5),  
#       axis.title = element_text(size = base_size),  
#       axis.text = element_text(size = base_size * 0.9),  
#       legend.title = element_text(size = base_size),  
#       legend.text = element_text(size = base_size * 0.9),  
#       panel.grid.major = element_line(linewidth = 0.4),  
#       panel.grid.minor = element_blank(),  
#       plot.margin = margin(10, 10, 10, 10)  
#     )  
# }
```

```
# theme_set(theme_paper())
```

```
[2]: #save.image("workspaces/afterCorrelations_wSTAR.RData")
load("workspaces/afterCorrelations_wSTAR_thr2percCells.RData")
rm(mat_stellarscope)
rm(mat_soloTE)
rm(mat_STARsolo)
gc()
```

		used	(Mb)	gc trigger	(Mb)	max used	(Mb)
A matrix: 2×6 of type dbl	Ncells	28204544	1506.3	43199228	2307.1	28218124	1507.1
	Vcells	249634450	1904.6	363616095	2774.2	298919446	2280.6

1 Gene Intersections

```
[3]: library(rtracklayer)
# import the annotation used for snakemake
gene_annotation <- rtracklayer::import("/mnt/TEdataStorage_2T/snakemake_data/
↳ references/mm10/gencode.vM10.annotation.gtf.gz")
names(mcols(gene_annotation))
```

1. 'source' 2. 'type' 3. 'score' 4. 'phase' 5. 'gene_id' 6. 'gene_type' 7. 'gene_status'
8. 'gene_name' 9. 'level' 10. 'havana_gene' 11. 'transcript_id' 12. 'transcript_type'
13. 'transcript_status' 14. 'transcript_name' 15. 'transcript_support_level' 16. 'tag' 17. 'ha-
vana_transcript' 18. 'exon_number' 19. 'exon_id' 20. 'protein_id' 21. 'ccdsid' 22. 'ont'

```
[4]: gene_annotation_df <- as.data.frame(gene_annotation)

# Extract only exon entries
exons <- gene_annotation[gene_annotation$type == "exon"]
names(exons) <- exons$gene_name
```

```
[5]: # keep only TEs that are in both SoloTE and Stellarscope
annotation_common <- na.omit(conversionTable)

cat("Classes not in SoloTE annotation: ",
    paste(setdiff(unique(conversionTable$class),
↳ unique(annotation_common$class)), collapse = ", "),
    "\n")
```

Classes not in SoloTE annotation: Other, Satellite, Unknown, NA, RNA

```
[6]: objTElist <- list()

objTElist[["SoloTE"]] <- objTE_soloTE
objTElist[["Stellarscope"]] <- objTE_stellarscope
```

```
objTElist[["STARsolo"]] <- objTE_STARsolo
```

```
objTElist
```

```
$SoloTE
```

```
An object of class Seurat
```

```
47725 features across 1542 samples within 1 assay
```

```
Active assay: RNA (47725 features, 4000 variable features)
```

```
3 layers present: counts, data, scale.data
```

```
2 dimensional reductions calculated: pca, umap
```

```
$Stellarscope
```

```
An object of class Seurat
```

```
38713 features across 1542 samples within 1 assay
```

```
Active assay: RNA (38713 features, 4000 variable features)
```

```
3 layers present: counts, data, scale.data
```

```
2 dimensional reductions calculated: pca, umap
```

```
$STARsolo
```

```
An object of class Seurat
```

```
37929 features across 1542 samples within 1 assay
```

```
Active assay: RNA (37929 features, 4000 variable features)
```

```
3 layers present: counts, data, scale.data
```

```
2 dimensional reductions calculated: pca, umap
```

```
[7]: # Compute overlaps to exons and save lists of loci
featuresList <- list()

bedList <- list()
grList <- list()
exonOverlapList <- list()
toolOverlappingGrList <- list()
toolOverlappingList <- list()

for(tool in names(objTElist)){
  if(tool == "SoloTE"){
    bedList[[tool]] <- conversionTable[conversionTable$soloteID %in%
↳Features(objTElist[[tool]]),
    c("chr","start","end","strand","stellarscopeID")]
    print(nrow(bedList[[tool]]))
  }else{
    bedList[[tool]] <- conversionTable[conversionTable$stellarscopeID %in%
↳Features(objTElist[[tool]]),
    c("chr","start","end","strand","stellarscopeID")]
    print(nrow(bedList[[tool]]))
  }
}
```

```

    bedList[[tool]] <- bedList[[tool]][!
↪duplicated(bedList[[tool]]$stellarscopeID),]
    bedList[[tool]] <- bedList[[tool]][!is.na(bedList[[tool]]$stellarscopeID),]
    featuresList[[tool]] <- bedList[[tool]]$stellarscopeID
    print(nrow(bedList[[tool]]))
    rownames(bedList[[tool]]) <- bedList[[tool]]$stellarscopeID
    # turn into GR object
    grList[[tool]] <- makeGRangesFromDataFrame(bedList[[tool]])
    exonOverlapList[[tool]] <- findOverlaps(grList[[tool]], exons, minoverlap =
↪10)

    toolOverlappingGrList[[tool]] <- grList[[tool]][unique(
↪exonOverlapList[[tool]]@from)]
    toolOverlappingList[[tool]] <- rownames(as.data.
↪frame(toolOverlappingGrList[[tool]]))
    #print(as.data.frame(toolOverlappingList[[tool]]))
}

```

[1] 47677

[1] 47668

[1] 38713

[1] 38713

Warning message in .merge_two_Seqinfo_objects(x, y):

"Each of the 2 combined objects has sequence levels not in the other:

- in 'x': GL456210.1, GL456211.1, GL456212.1, GL456216.1, GL456221.1,
GL456233.1, GL456350.1, GL456354.1, GL456368.1, GL456372.1, JH584299.1,
JH584304.1

- in 'y': chrM

Make sure to always combine/compare objects based on the same reference
genome (use suppressWarnings() to suppress this warning)."

[1] 37929

[1] 37929

Warning message in .merge_two_Seqinfo_objects(x, y):

"Each of the 2 combined objects has sequence levels not in the other:

- in 'x': GL456210.1, GL456211.1, GL456212.1, GL456216.1, GL456219.1,
GL456221.1, GL456233.1, GL456350.1, GL456354.1, GL456366.1, GL456367.1,
GL456368.1, GL456372.1, GL456378.1, GL456393.1, JH584293.1, JH584294.1,
JH584296.1, JH584297.1, JH584298.1, JH584299.1, JH584304.1

- in 'y': chrM

Make sure to always combine/compare objects based on the same reference
genome (use suppressWarnings() to suppress this warning)."

[8]: colorTools

SoloTE

'#A4DD9B' Stellarscope

'#4f5d93' STARsolo

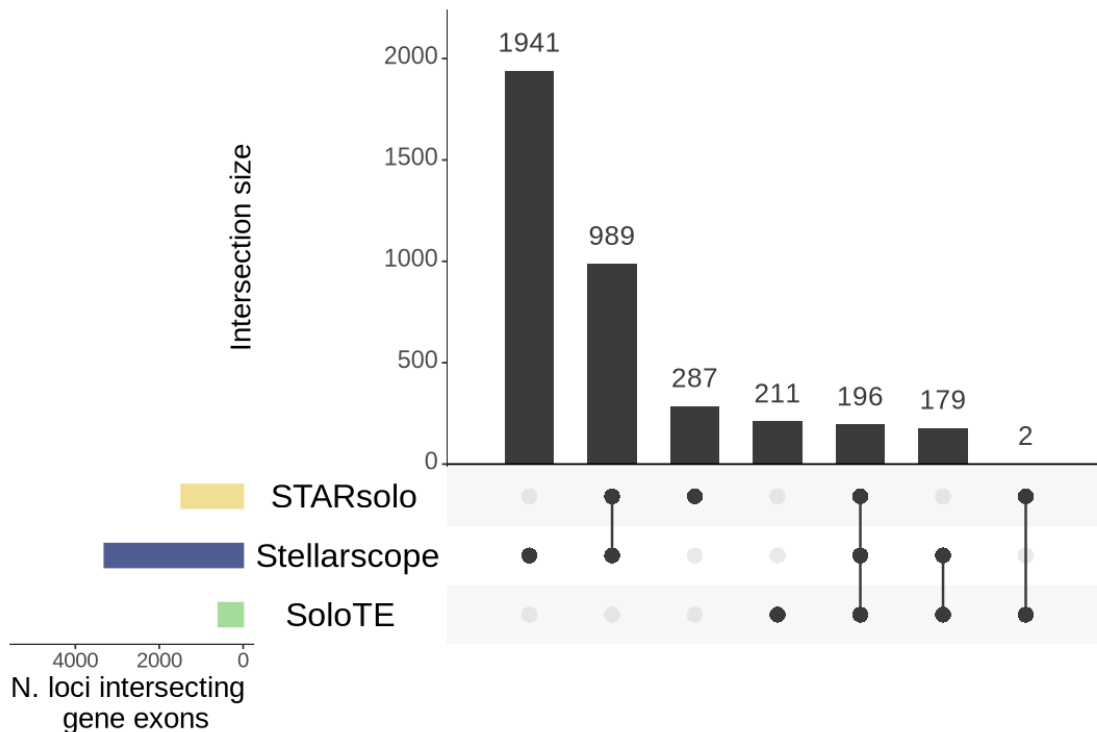
'#f0df93'

```
[64]: library(UpSetR)
options(repr.plot.width=9, repr.plot.height=6)

ph <- UpSetR::upset(fromList(toolOverlappingList),
  sets=names(toolOverlappingList), sets.bar.color=colorTools,
  keep.order = TRUE,
  text.scale = c(2, 2, 2, 1.75, 2.75, 2.5),
  order.by=c("freq"),
  point.size=4, mb.ratio = c(0.63, 0.37),
  set_size.show=F, set_size.scale_max=
  ↪max(lengths(toolOverlappingList))+2000,
  sets.x.label="N. loci intersecting \n gene exons",
  mainbar.y.label="Intersection size")

pdf(paste0("figures_", dataset_id, "/NlociIntersectingGeneExons_upset.pdf"),
  ↪width=9, height=6)
ph
dev.off()
ph
```

pdf: 2



```
[15]: df <- NULL

for(tool in names(objTElist)){

  loci <- bedList[[tool]]$stellarscopeID
  lociOverlappingGenes <- toolOverlappingList[[tool]]

  df <- rbind(df, c(tool, length(loci), length(lociOverlappingGenes)))

}

df <- as.data.frame(df)
df
```

	V1	V2	V3
	<chr>	<chr>	<chr>
A data.frame: 3 × 3	SoloTE	47668	588
	Stellarscope	38713	3305
	STARsolo	37929	1474

```
[16]: library(dplyr)
library(purrr)
library(tidyr)

make_membership_table <- function(all_lists, gene_lists=NULL) {

  # All loci that appear in at least one tool
  all_loci <- unique(unlist(all_lists))

  # Presence/absence table (automatic)
  df <- tibble(locus = all_loci)

  for (nm in names(all_lists)) {
    df[[nm]] <- df$locus %in% all_lists[[nm]]
  }

  # Optional: add gene-overlap column
  if (!is.null(gene_lists)) {
    gene_loci <- unique(unlist(gene_lists))
    df$gene_overlap <- df$locus %in% gene_loci
  }

  df
}

df <- make_membership_table(featuresList, toolOverlappingList)
head(df)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:GenomicRanges':

intersect, setdiff, union

The following object is masked from 'package:GenomeInfoDb':

intersect

The following objects are masked from 'package:IRanges':

collapse, desc, intersect, setdiff, slice, union

The following objects are masked from 'package:S4Vectors':

first, intersect, rename, setdiff, setequal, union

The following objects are masked from 'package:BiocGenerics':

combine, intersect, setdiff, union

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

Attaching package: 'purrr'

The following object is masked from 'package:GenomicRanges':

reduce

The following object is masked from ‘package:IRanges’:

reduce

Attaching package: ‘tidyr’

The following object is masked from ‘package:S4Vectors’:

expand

A tibble: 6 × 5

locus <chr>	SoloTE <lgl>	Stellarscope <lgl>	STARsolo <lgl>	gene_overlap <lgl>
MT2-Mm-dup2	TRUE	TRUE	TRUE	FALSE
MLT1A-dup2	TRUE	TRUE	TRUE	FALSE
L1-Mus4-dup3	TRUE	TRUE	TRUE	FALSE
MT2-Mm-dup3	TRUE	TRUE	TRUE	FALSE
L1Md-F2-dup62	TRUE	TRUE	TRUE	FALSE
L1MC2-dup3	TRUE	TRUE	FALSE	FALSE

```
[17]: df <- df %>%  
  mutate(  
    group = pmap_chr(select(., SoloTE:STARsolo), ~ {  
      here <- c(...)  
      tools <- names(here)[here]  
      if (length(tools) == 0) return(NA_character_)  
      paste(tools, collapse = "&")  
    })  
  )
```

```
[18]: summary_df <- df %>%  
  group_by(group) %>%  
  summarise(  
    N = n(),  
    gene_N = sum(gene_overlap),  
    percent_gene = round(100 * gene_N / N, 1),  
    .groups = "drop"  
  ) %>%  
  arrange(desc(N))
```

```
[19]: lengths(toolOverlappingList)  
summary_df
```

SoloTE	588 Stellarscope	3305 STARsolo			1474
	group <chr>	N <int>	gene_N <int>	percent_gene <dbl>	
A tibble: 7 × 4	STARsolo	21035	287	1.4	
	SoloTE	19726	211	1.1	
	SoloTE&Stellarscope	17215	179	1.0	
	SoloTE&Stellarscope&STARsolo	10630	196	1.8	
	Stellarscope&STARsolo	6167	989	16.0	
	Stellarscope	4701	1941	41.3	
	SoloTE&STARsolo	97	2	2.1	

```
[20]: mix_colors <- function(cols) {
  cols_rgb <- t(col2rgb(cols))
  avg_rgb <- colMeans(cols_rgb)
  rgb(avg_rgb[1], avg_rgb[2], avg_rgb[3], maxColorValue = 255)
}

library(purrr)

# 1-way (original tool colors)
color_1way <- colorTools

# 2-way mixes
color_2way <- list(
  "SoloTE&Stellarscope" = mix_colors(colorTools[c("SoloTE", "Stellarscope")]),
  "SoloTE&STARsolo"     = mix_colors(colorTools[c("SoloTE", "STARsolo")]),
  "Stellarscope&STARsolo" = mix_colors(colorTools[c("Stellarscope", "STARsolo")])
)

# 3-way mix
color_3way <- c(
  "SoloTE&Stellarscope&STARsolo" =
    mix_colors(colorTools[c("SoloTE", "Stellarscope", "STARsolo")])
)

# Combine everything
intersection_colors <- c(color_1way, color_2way, color_3way)
intersection_colors
```

```
$SoloTE '#A4DD9B'
$Stellarscope '#4f5d93'
$STARsolo '#f0df93'
$'SoloTE&Stellarscope' '#799D97'
$'SoloTE&STARsolo' '#CADE97'
$'Stellarscope&STARsolo' '#9F9E93'
```

\$'SoloTE&Stellarscope&STARsolo' '#A1B395'

[21]: summary_df

	group <chr>	N <int>	gene_N <int>	percent_gene <dbl>
A tibble: 7 × 4	STARsolo	21035	287	1.4
	SoloTE	19726	211	1.1
	SoloTE&Stellarscope	17215	179	1.0
	SoloTE&Stellarscope&STARsolo	10630	196	1.8
	Stellarscope&STARsolo	6167	989	16.0
	Stellarscope	4701	1941	41.3
	SoloTE&STARsolo	97	2	2.1

```
[48]: options(repr.plot.width=9, repr.plot.height=4)
library(ggpubr)

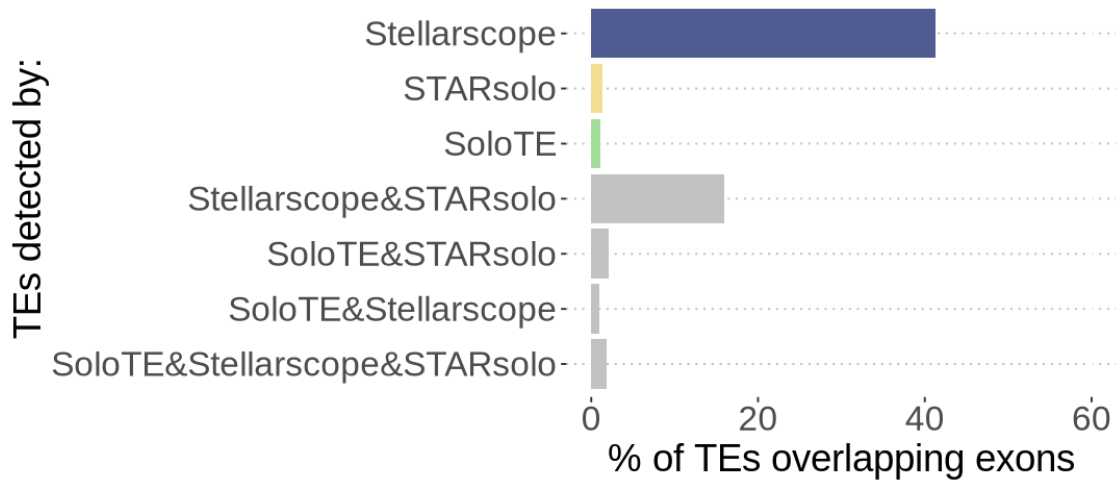
df <- summary_df
# Step 1: compute size
df$size <- sapply(df$group, function(x) length(strsplit(x, "&")[[1]]))

# Step 2: get **unique groups** and sort by size
unique_groups <- unique(df$group)
group_levels <- unique_groups[order(sapply(unique_groups, function(x)
  ↪length(strsplit(x, "&")[[1]])))]

# descending within each size
df <- df %>%
  arrange(-size, percent_gene) # or arrange(size, percent) for ascending

df$group <- factor(df$group, levels = df$group) # preserve new order

ggplot(df, aes(y = group, x = percent_gene, fill = group)) +
  geom_col(alpha=1) +
  scale_fill_manual(values = colorTools, na.value = "grey76") +
  xlab("% of TEs overlapping exons") +
  ylab("TEs detected by:") +
  xlim(0,60) +
  theme_pubclean() +
  theme(text=element_text(size=22),
        axis.text=element_text(size=20),
        legend.position = "none"
  )
ggsave(paste0("figures_", dataset_id, "/percTEsOverlappingExons.pdf"), width=8,
  ↪height=6)
```



```
[51]: options(repr.plot.width=9, repr.plot.height=4)

df <- summary_df
# Step 1: compute size
df$size <- sapply(df$group, function(x) length(strsplit(x, "&")[[1]]))

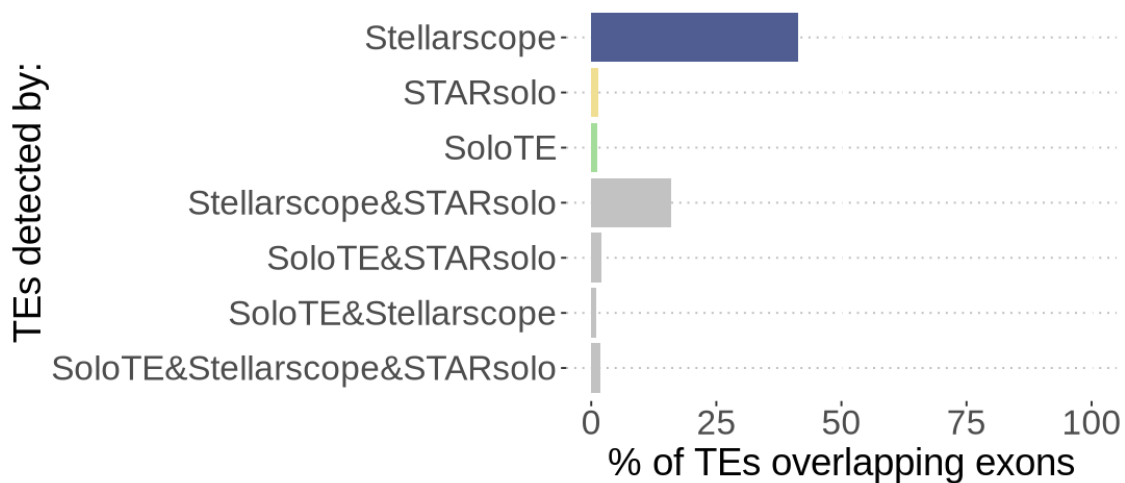
# Step 2: get **unique groups** and sort by size
unique_groups <- unique(df$group)
group_levels <- unique_groups[order(sapply(unique_groups, function(x)
  length(strsplit(x, "&")[[1]])))]

# descending within each size
df <- df %>%
  arrange(-size, percent_gene) # or arrange(size, percent) for ascending

df$group <- factor(df$group, levels = df$group) # preserve new order

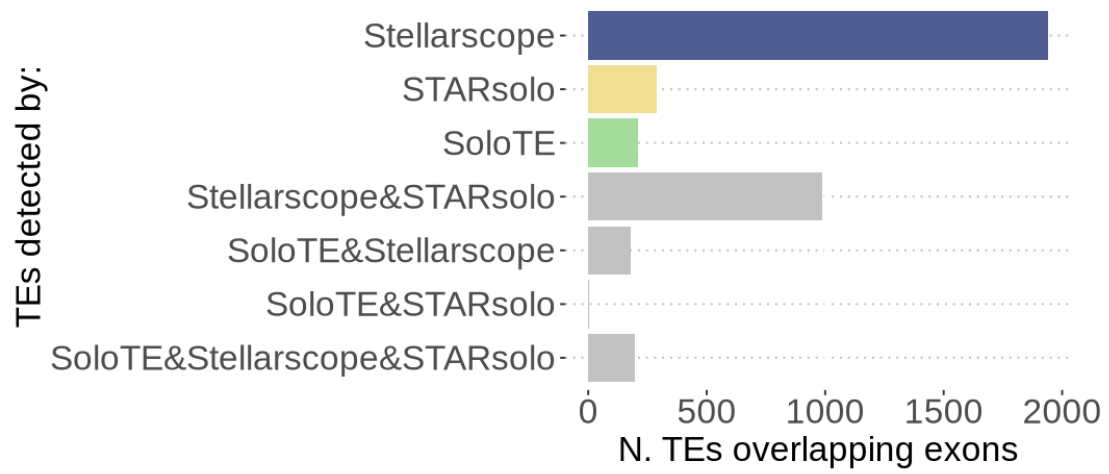
ggplot(df, aes(y = group, x = percent_gene, fill = group)) +
  geom_col() +
  scale_fill_manual(values = colorTools, na.value = "grey76") +
  xlab("% of TEs overlapping exons") +
  ylab("TEs detected by:") +
  xlim(0,100) +
  theme_pubclean() +
  theme(text=element_text(size=22),
        axis.text=element_text(size=20),
        legend.position = "none"
  )
```

```
ggsave(paste0("figures_", dataset_id, "/percTEsOverlappingExons_max100.pdf"),
  width=8, height=4)
```



```
[50]: # descending within each size
df <- df %>%
  arrange(-size, gene_N) # or arrange(size, percent) for ascending

df$group <- factor(df$group, levels = df$group) # preserve new order
ggplot(df, aes(y = group, x = gene_N, fill = group)) +
  geom_col() +
  scale_fill_manual(values = colorTools, na.value = "grey76") +
  xlab("N. TEs overlapping exons") +
  ylab("TEs detected by:") +
  theme_pubclean() +
  theme(text=element_text(size=20),
        axis.text=element_text(size=20),
        legend.position = "none",
        plot.margin = margin(t = 10, r = 30, b = 10, l = 10) # top, right,
  bottom, left
  )
ggsave(paste0("figures_", dataset_id, "/nTEsOverlappingExons.pdf"), width=8,
  height=4)
```



[]: